

# **Time Series Analysis with Python**

# Action 01: Search for Colab

The screenshot shows a web browser window with two tabs. The first tab is 'YouTube' and the second is 'google colab - Search'. The address bar shows the URL: <https://www.bing.com/search?q=google+colab&form=ANNT1&ref=6a9f9e80bb1a47...>. The search bar contains the text 'google colab'. Below the search bar, there are tabs for 'ALL', 'SEARCH', 'IMAGES', 'VIDEOS', 'MAPS', 'NEWS', and 'MORE'. The 'ALL' tab is selected. The search results show 'About 77,200 results'. The first result is 'Google Colab' with the URL <https://colab.research.google.com> and a link to 'Translate this result'. Below the result, there are four links: 'Google Colab', 'Welcome To Colab!', 'Colab FAQ', 'Python', 'Guide to Markdown', and 'Differences between Colab Markdown and other Markdown dialects Colab uses ...'.

YouTube

google colab - Search

<https://www.bing.com/search?q=google+colab&form=ANNT1&ref=6a9f9e80bb1a47...>

google colab

ALL SEARCH IMAGES VIDEOS MAPS NEWS MORE

About 77,200 results

Google Colab  
<https://colab.research.google.com> [Translate this result](#)

**Google Colab**  
Learn how to bring the power of Colab straight to your terminal. Read the full announcement.

**Welcome To Colab!**  
Colab notebooks allow you to combine executable code and rich text in a single ...

**Colab FAQ**  
Colab notebooks are stored in Google Drive, or can be loaded from GitHub. Colab ...

**Python**  
Note: This notebook uses the Interactions API, the latest way to interact with ...

**Guide to Markdown**  
Differences between Colab Markdown and other Markdown dialects Colab uses ...

# Action 02: Open Colab

The screenshot shows a web browser with three tabs: YouTube, google colab - Search, and Welcome To Colab - Colab. The address bar displays the URL [https://colab.research.google.com/notebooks/intro.ipynb#scrollTo=5fCEDCU\\_qrC0](https://colab.research.google.com/notebooks/intro.ipynb#scrollTo=5fCEDCU_qrC0). The Colab interface features a top bar with a search icon, 'Commands', a circled '+ Code' button, '+ Text', and a 'Run all' button. On the left, a sidebar contains icons for a menu, search, code editor, and file explorer. The 'Table of contents' panel lists: 'Welcome to Colab!' (highlighted), 'Getting started', 'Google Colab is available in V...', 'Access popular AI model...', 'Explore the Gemini API', 'Data science', 'Machine learning', and 'More Resources'. The main content area displays 'Welcome to Colab!' with a sub-section 'What is Colab?' explaining that Colab, or 'Colaboratory', allows running Python in the browser with zero configuration and free GPU access.

Commands + Code + Text ▶ Run all

Table of contents

- Welcome to Colab!
- Getting started
- Google Colab is available in V...
- Access popular AI model...
- Explore the Gemini API
- Data science
- Machine learning
- More Resources

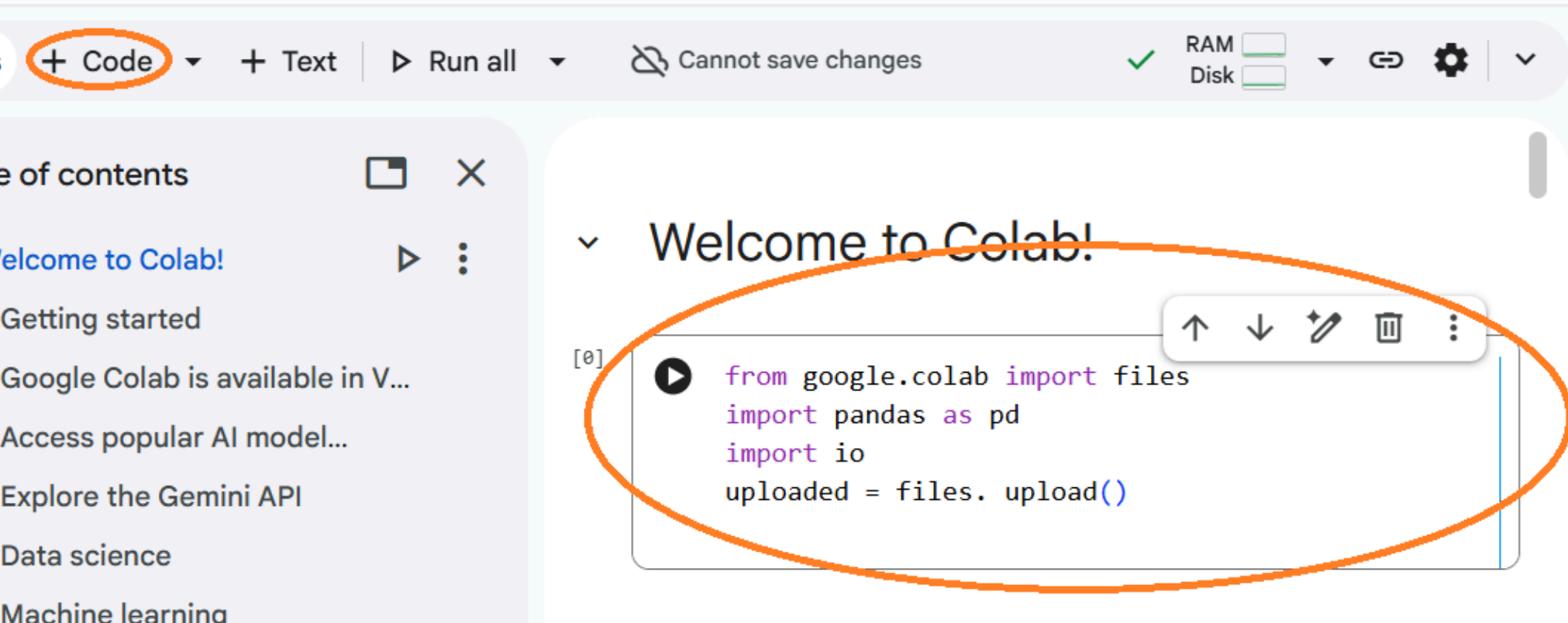
## Welcome to Colab!

### What is Colab?

Colab, or "Colaboratory", allows you to run Python in your browser, with

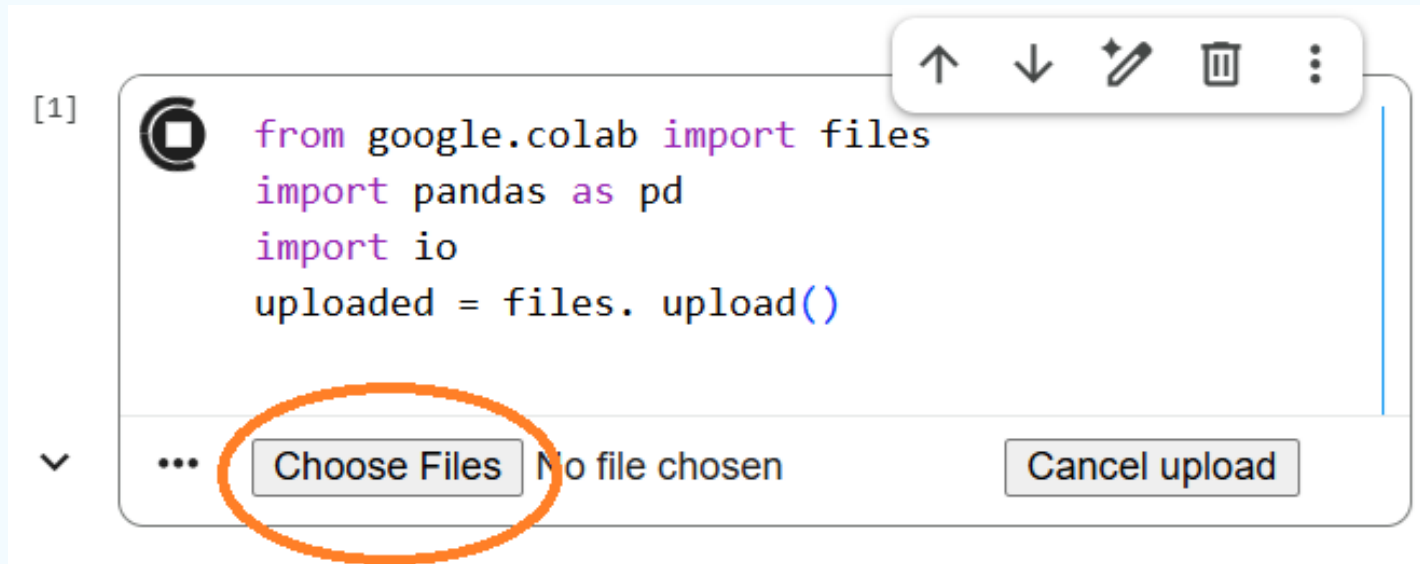
- Zero configuration required
- Access to GPUs free of charge

# Action 03: Upload files to Colab

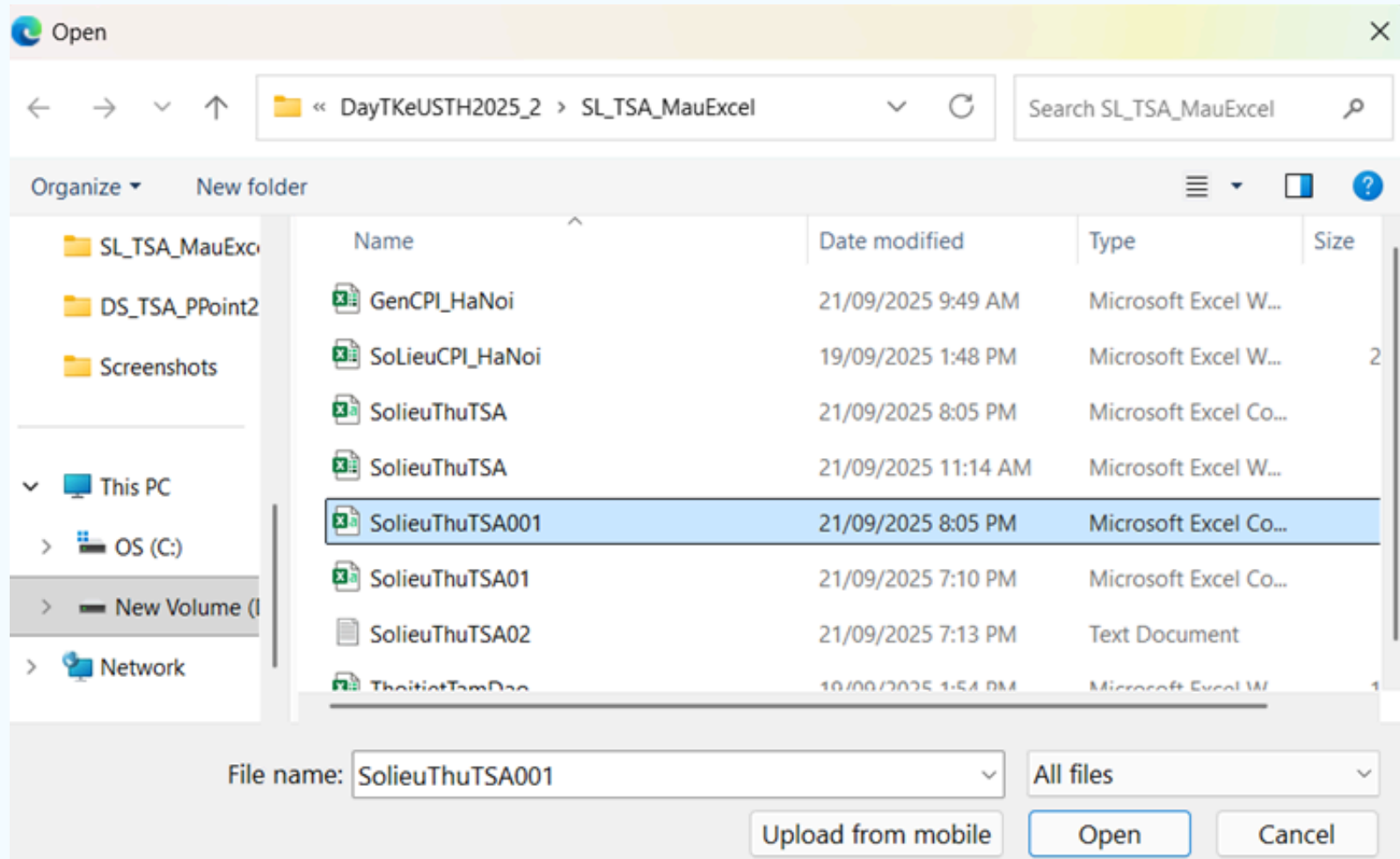


```
from google.colab import files
import pandas as pd
import io
uploaded = files.upload()
```

## Action 03: Upload files to Colab



# Action 03: Upload files to Colab



## Action 03: Upload files to Colab



```
from google.colab import files  
import pandas as pd  
import io  
uploaded = files.upload()
```



SolieuThuTSA001.csv

**SolieuThuTSA001.csv**(text/csv) - 3415 bytes, last modified: 9/21/2025 - 100% done  
Saving SolieuThuTSA001.csv to SolieuThuTSA001.csv

## Action 04: Print file

```
▶ df = pd.read_csv('SolieuThuTSA001.csv')  
print(df.head(12))
```

| ↔  | Time | Year | Month | Consumer price index | CPI_next   | CPI_rate  |
|----|------|------|-------|----------------------|------------|-----------|
| 0  | 1    | 2015 | 1     | 98.8                 | 98.800000  | -1.200000 |
| 1  | 2    | 2015 | 2     | 99.3                 | 100.506073 | 0.506073  |
| 2  | 3    | 2015 | 3     | 100.4                | 101.107754 | 1.107754  |
| 3  | 4    | 2015 | 4     | 100.2                | 99.800797  | -0.199203 |
| 4  | 5    | 2015 | 5     | 100.1                | 99.900200  | -0.099800 |
| 5  | 6    | 2015 | 6     | 100.1                | 100.000000 | 0.000001  |
| 6  | 7    | 2015 | 7     | 100.2                | 100.099900 | 0.099900  |
| 7  | 8    | 2015 | 8     | 100.2                | 100.000000 | 0.000001  |
| 8  | 9    | 2015 | 9     | 99.9                 | 99.700599  | -0.299401 |
| 9  | 10   | 2015 | 10    | 100.1                | 100.200200 | 0.200200  |
| 10 | 11   | 2015 | 11    | 100.0                | 99.900100  | -0.099900 |
| 11 | 12   | 2015 | 12    | 100.0                | 100.000000 | 0.000001  |

```
df = pd.read_csv('SolieuThuTSA001.csv')  
print(df.head(12))
```



## Action 05: Create plot of time series

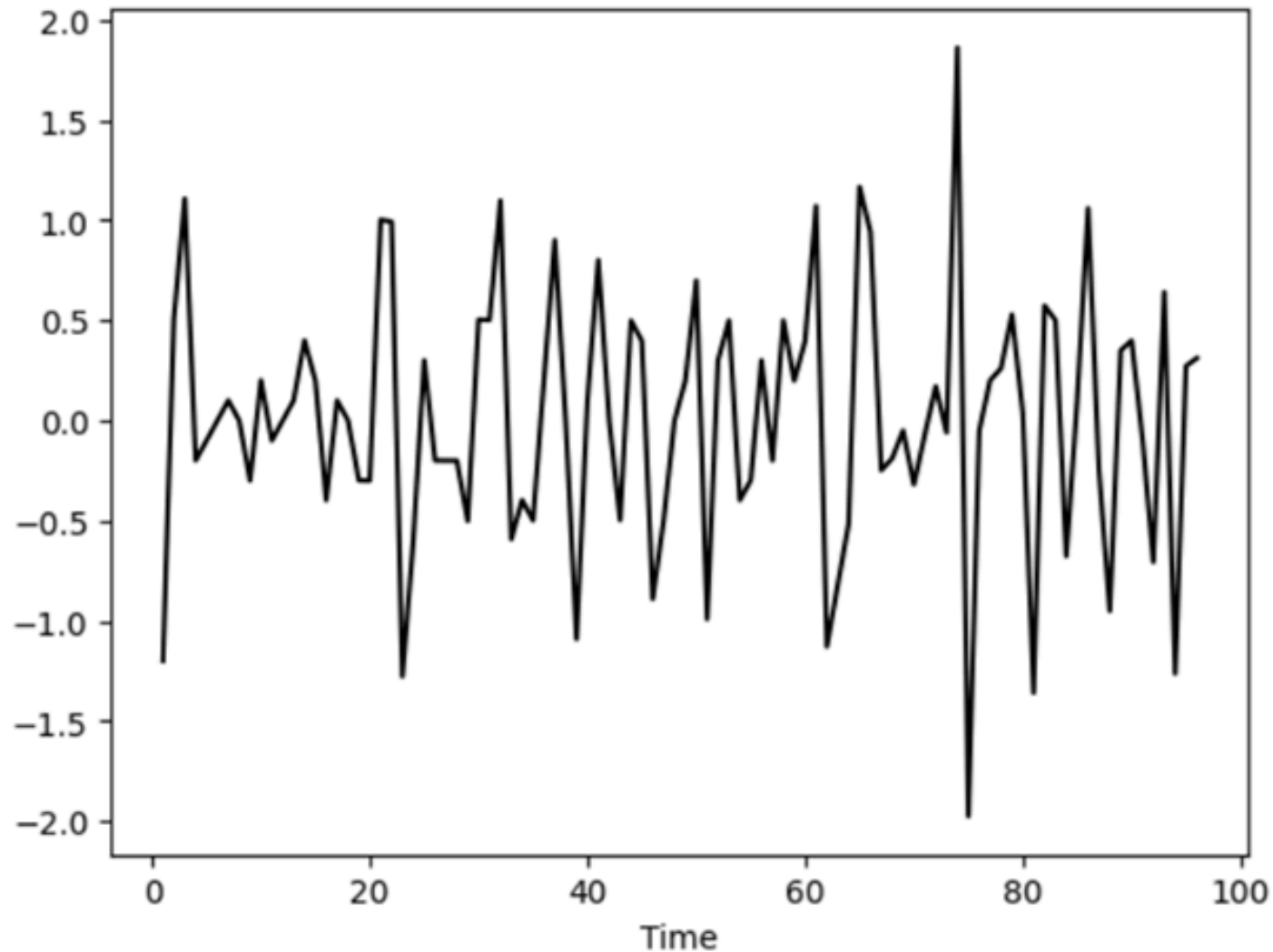
```
import matplotlib.pyplot as plt
from datetime import datetime
fig = plt.figure()
ax = fig.add_subplot(1, 1, 1)
data = pd.read_csv('SolieuThuTSA001.csv', index_col=0, parse_dates=True)
next = data['CPI_rate']
next.plot(ax=ax, style='k-')
```

```
import matplotlib.pyplot as plt
from datetime import datetime
fig = plt.figure()
ax = fig.add_subplot(1,1,1)
data = pd.read_csv('SolieuThuTSA001.csv')
next = data['CPI_rate']
next.plot(ax=ax, style='k-')
```

## Action 05: Create plot of time series



```
/tmp/ipython-input-2133552139.py:5: UserWarning: Could not infer format, so each
data = pd.read_csv('SolieuThuTSA001.csv', index_col=0, parse_dates=True)
<Axes: xlabel='Time'>
```

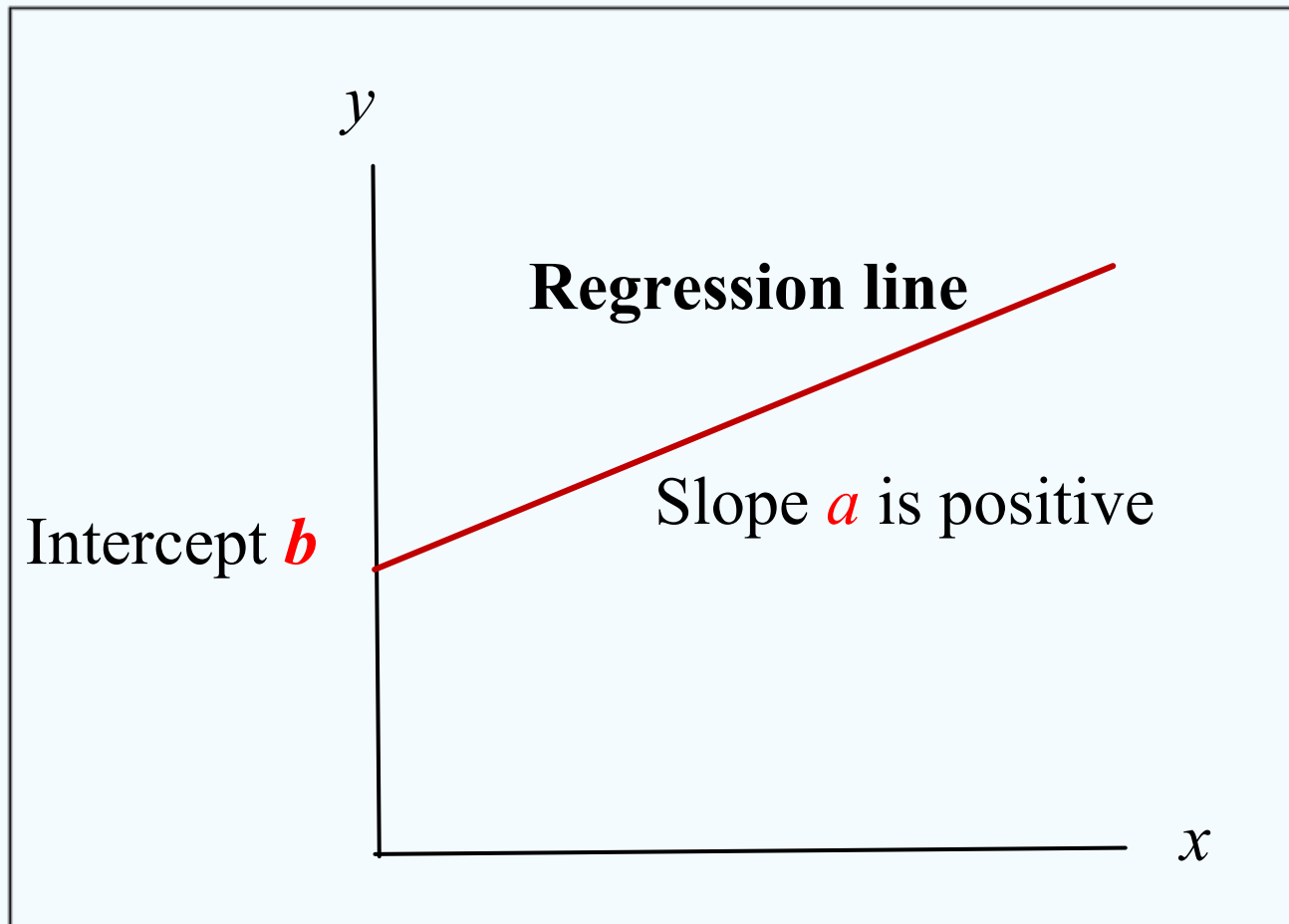


## Action 06: Trend checking by regression analysis

- \*  $b$  is called *regression constant* (*intercept*), showing the *intersection* point of regression line and vertical axis, that is the value of  $Y$  when  $X$  takes value  $0$ ;
- \*  $e$  is *residual* of regression, indicates error of estimation at each point of observation.

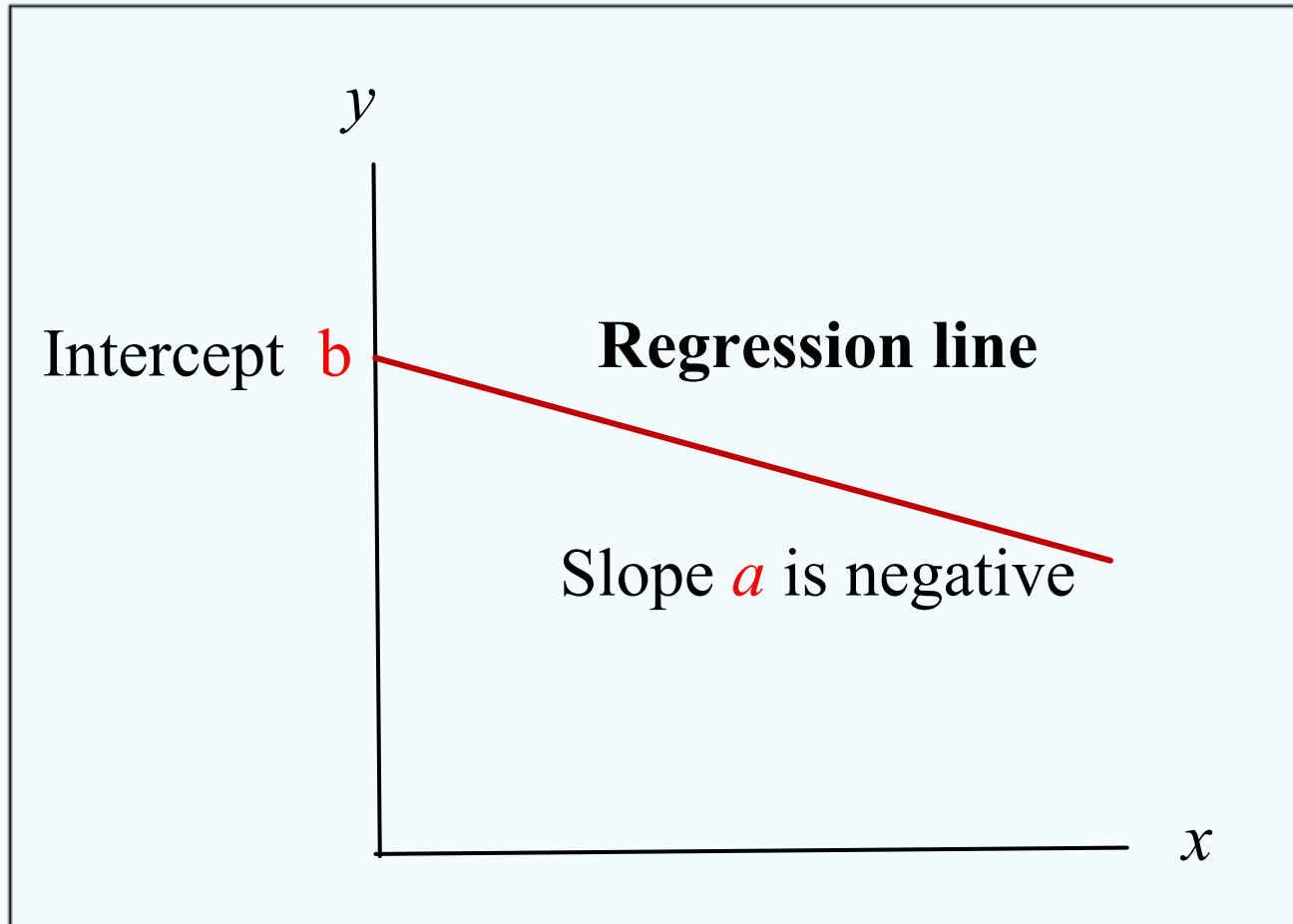
# Simple Linear Regression Equation

- Positive Linear Relationship



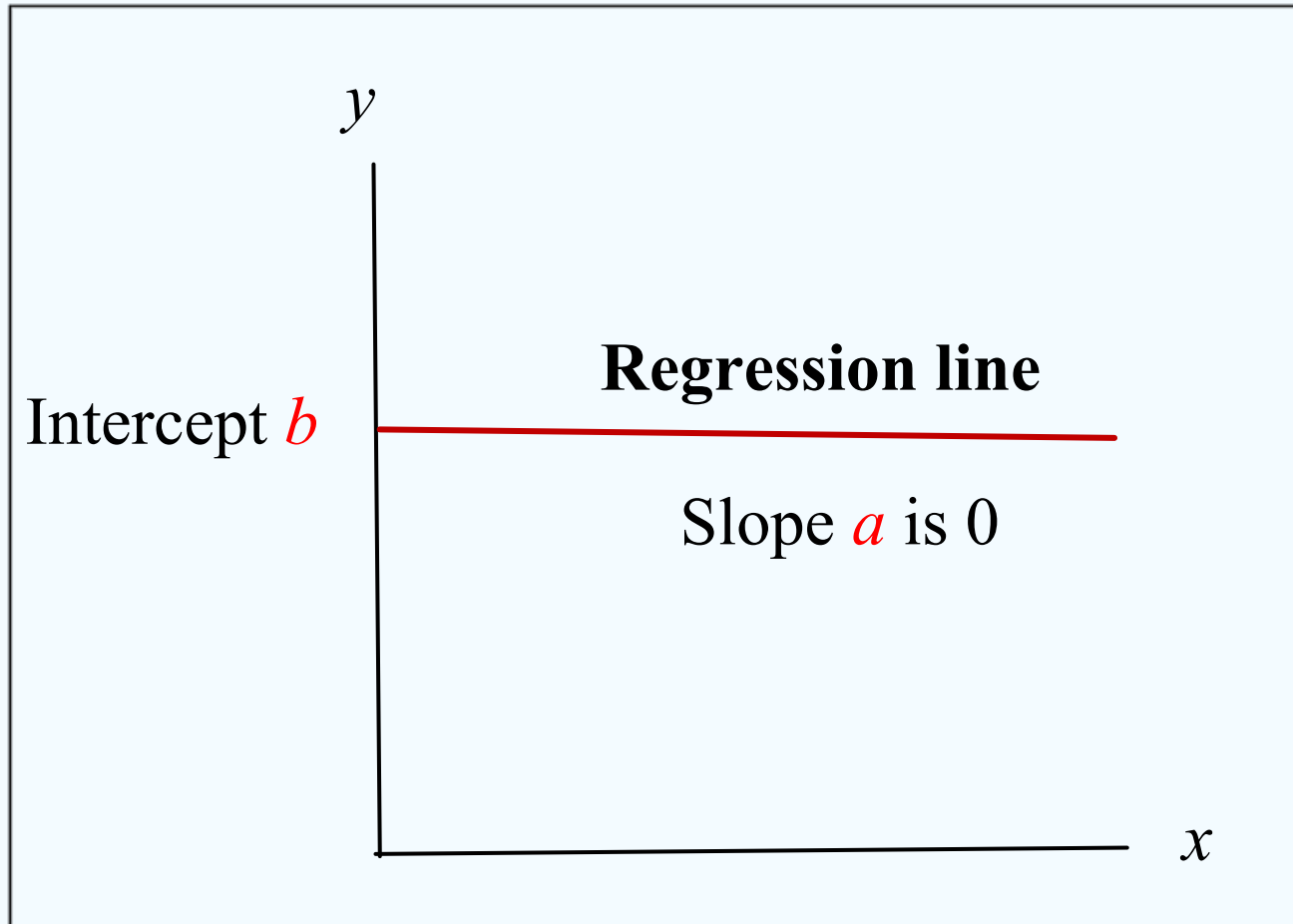
# Simple Linear Regression Equation

- Negative Linear Relationship



# Simple Linear Regression Equation

- No Relationship





- Compare the p-value to the significance level  $\alpha$ .
- If  $p \geq \alpha$   Accept the hypothesis, consider  $a = 0$ , between Y and X *there is no linear relationship*, increase of X *does not lead* Y to increase or decrease, *there is no trend*.
- If  $p < \alpha$   Reject the hypothesis, assert  $a \neq 0$ , the conclusion between Y and X really has a strong linear relationship, the *increase* in the value of X will lead to an *increase* (if  $a > 0$ , *positive trend*), or *reduce* (if  $a < 0$ , *negative trend*) the value of Y.
- The test for intercept coefficient  $b$  is also carried out in the same way.



# Confidence Interval for $a$ and $b$

- The form of a confidence interval for  $a$  and  $b$  are:

$$(a^{\wedge} - t_{1-\alpha/2}sa^{\wedge}; a^{\wedge} + t_{1-\alpha/2}sa^{\wedge}) \quad (b^{\wedge} - t_{1-\alpha/2}sb^{\wedge}; b^{\wedge} + t_{1-\alpha/2}sb^{\wedge})$$

where

$a^{\wedge}$  and  $b^{\wedge}$  are the point estimators,

$t_{1-\alpha/2}sa^{\wedge}$  and  $t_{1-\alpha/2}sb^{\wedge}$  are the margins of error,

$t_{1-\alpha/2}$  is the  $t$  value providing an area of  $\alpha/2$  in the upper tail of a  $t$  distribution with  $n - 2$  degrees of freedom.

# Confidence Interval for $a$ (or $b$ )

- We can use a confidence interval for  $a$  (or  $b$ ) to test the hypotheses just used in the  $t$  test.
- **$H$  is rejected** if  $0$  (the hypothesized value of  $a$  (or  $b$ )) is not included in the confidence interval for  $a$  (or  $b$ , respectively) .

## Action 06: Trend checking by regression analysis

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn import preprocessing
```

```
import statsmodels.formula.api as smf
lm = smf.ols(formula='CPI_rate ~ Time', data=df).fit()
print(lm.params)
```

```
⇒ Intercept    0.017790
   Time        -0.000236
   dtype: float64
```

# Action 06: Trend checking by regression analysis

lm.summary()

OLS Regression Results

|                   |                  |                     |          |
|-------------------|------------------|---------------------|----------|
| Dep. Variable:    | CPI_rate         | R-squared:          | 0.000    |
| Model:            | OLS              | Adj. R-squared:     | -0.011   |
| Method:           | Least Squares    | F-statistic:        | 0.009868 |
| Date:             | Mon, 22 Sep 2025 | Prob (F-statistic): | 0.921    |
| Time:             | 08:02:43         | Log-Likelihood:     | -93.198  |
| No. Observations: | 96               | AIC:                | 190.4    |
| Df Residuals:     | 94               | BIC:                | 195.5    |
| Df Model:         | 1                |                     |          |

Covariance Type: nonrobust

|           | coef    | std err | t      | P> t  | [0.025 | 0.975] |
|-----------|---------|---------|--------|-------|--------|--------|
| Intercept | 0.0178  | 0.133   | 0.134  | 0.894 | -0.246 | 0.281  |
| Time      | -0.0002 | 0.002   | -0.099 | 0.921 | -0.005 | 0.004  |

Omnibus: 2.566 Durbin-Watson: 2.251  
Prob(Omnibus): 0.277 Jarque-Bera (JB): 2.019  
Skew: -0.201 Prob(JB): 0.364  
Kurtosis: 3.586 Cond. No. 113.

H: slope = 0

P-value = 0.921 > 5%

- Accept H
- Conclude:  
Trend not exists

95% CI of slope  
= (-0.005;0.004)  
contains 0

- Accept H
- Conclude:  
Trend not exists

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,
mean_absolute_error
from sklearn import preprocessing
```

```
import statsmodels.formula.api as smf
lm = smf.ols(formula='CPI_rate ~ Time', data=df).fit()
print(lm.params)
```

```
lm.summary()
```

## Action 07: Stationarity checking

Most classical and deep learning time series models (ARMA, VAR, Prophet, LSTM) assume that the data is stationary. Non-stationary data can lead to:

- Spurious regressions
- Poor model accuracy
- Invalid statistical inferences

# Action 07: Stationarity checking

## Augmented Dickey-Fuller test

This is a statistical test that is dedicatedly built to test whether univariate time series data is stationary or not. Let's define the null and alternate hypotheses,

- **Ho (Null Hypothesis):** The time series data is non-stationary
  - **H1 (alternate Hypothesis):** The time series data is stationary
- Assume **alpha = 0.05**, meaning (95% confidence). The test results are interpreted with a p-value if  $p > 0.05$  fails to reject the null hypothesis, else if  $p \leq 0.05$  reject the null hypothesis.

## Action 07: Stationarity checking

```
▶ from google.colab import files  
import pandas as pd  
import io  
uploaded = files.upload()
```



Choose Files SolieuThuTSA001.csv

**SolieuThuTSA001.csv**(text/csv) - 3415 bytes, last modified: 9/21/2025 - 100% done  
Saving SolieuThuTSA001.csv to SolieuThuTSA001.csv

```
▶ data = pd.read_csv('SolieuThuTSA001.csv')
```

```
CPI_rate = data['CPI_rate']
```

```
from statsmodels.tsa.stattools import adfuller
```



## Action 07: Stationarity checking

```
▶ result = adfuller(CPI_rate)

# Extract and display results
print("ADF Statistic:", result[0])
print("p-value:", result[1])
print("Critical Values:")
for key, value in result[4].items():
    print(f"    {key}: {value}")

# Interpretation
if result[1] <= 0.05:
    print("The time series is stationary (reject the null hypothesis).")
else:
    print("The time series is not stationary (fail to reject the null hypothesis).")
```

## Action 07: Stationarity checking



ADF Statistic: -7.446385614920216

p-value: 5.823387502646691e-11

Critical Values:

1%: -3.50434289821397

5%: -2.8938659630479413

10%: -2.5840147047458037

The time series is stationary (reject the null hypothesis).

```
CPI_rate = data['CPI_rate']  
from statsmodels.tsa.stattools import adfuller  
result = adfuller(time_series)
```

```
# Extract and display results  
print("ADF Statistic:", result[0])  
print("p-value:", result[1])  
print("Critical Values:")  
for key, value in result[4].items():  
    print(f"  {key}: {value}")
```

```
# Interpretation  
if result[1] <= 0.05:  
    print("The time series is stationary (reject the null hypothesis).")  
else:  
    print("The time series is not stationary (fail to reject the null hypothesis).")
```

## Action 08: Calculating Autocorrelation

Autocorrelation measures the degree of **similarity** between a time series and a **lagged version** of itself over successive time intervals. When the autocorrelation in a time series is high, it becomes easier to predict future values by referring to past values.

To calculate autocorrelation in Python, you can use the acf function from the statsmodels library. Here is an example:

```
import statsmodels.api as sm
x = [x1, x2, ..., x51]

# Calculate autocorrelations
autocorrelations = sm.tsa.acf(x)
print(autocorrelations)
```

## Action 08: Calculating Autocorrelation

```
▶ import statsmodels.api as sm
```

```
▶ autocorrelations = sm.tsa.acf(CPI_rate)  
print(autocorrelations)
```

```
⇒ [ 1.          -0.14507382 -0.37821311 -0.06227817  0.06322561  0.01057099  
    0.02987517 -0.02350033  0.09350733 -0.01987309 -0.12177695  0.0145742  
    0.05490468  0.16889682 -0.21632196 -0.06750234  0.10298081  0.00807711  
   -0.0239603   0.12324056]
```

```
import statsmodels.api as sm
```

```
autocorrelations = sm.tsa.acf(x)  
print(autocorrelations)
```

## Action 08: Calculating Autocorrelation

The output will be an array of autocorrelation values for different lags. You can also specify the number of lags to use with the `nlags` argument:

```
autocorrelations = sm.tsa.acf(x, nlags=5)
print(autocorrelations)
```

```
▶ autocorrelations = sm.tsa.acf(CPI_rate, nlags=12)
print(autocorrelations)
```

```
⇒ [ 1.          -0.14507382 -0.37821311 -0.06227817  0.06322561  0.01057099
    0.02987517 -0.02350033  0.09350733 -0.01987309 -0.12177695  0.0145742
    0.05490468]
```

# Action 08: Calculating Autocorrelation

## Plotting Autocorrelation

To visualize the autocorrelation function, you can use the `plot_acf` function from the `statsmodels` library:

```
from statsmodels.graphics import tsaplots
import matplotlib.pyplot as plt
# Plot autocorrelation function
fig = tsaplots.plot_acf(CPI_rate, lags=50)
plt.show()
```

```
from statsmodels.graphics import tsaplots
import matplotlib.pyplot as plt
```

```
# Plot autocorrelation function
fig = tsaplots.plot_acf(CPI_rate, lags=50)
plt.show()
```

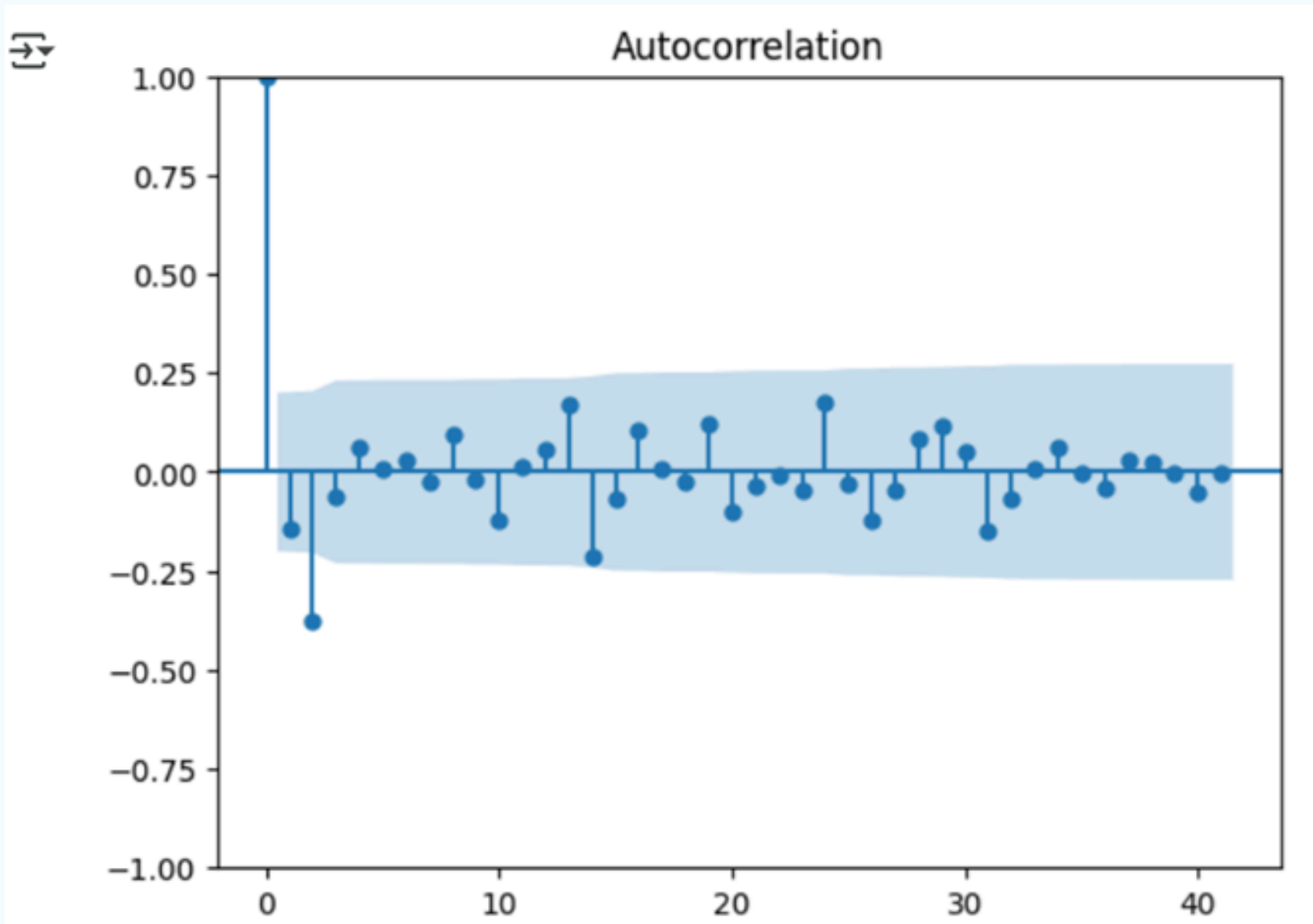
## Action 08: Calculating Autocorrelation

```
▶ from statsmodels.graphics import tsaplots
import matplotlib.pyplot as plt

# Plot autocorrelation function
fig = tsaplots.plot_acf(CPI_rate, lags=41)
plt.show()
```



## Action 08: Calculating Autocorrelation



## Action 09: Calculating Partial Autocorrelation

The **Partial Autocorrelation Function (PACF)** is a crucial tool in time series analysis that measures the partial correlation between a stationary time series and its own lagged values, considering and accounting for the values at all shorter lags. This is distinct from the Autocorrelation Function (ACF), which does not factor in the influence of other lags.

```
import pandas as pd
import numpy as np
from statsmodels.tsa.stattools import pacf
from statsmodels.graphics.tsaplots import plot_pacf
import matplotlib.pyplot as plt
```

## Action 09: Calculating Partial Autocorrelation

```
data = pd.read_csv('SolieuThuTSA001.csv')
CPI_rate = data['CPI_rate']
```

```
# Computing and Plotting PACF
pacf_values = pacf(CPI_rate, nlags=12)
print("Partial Autocorrelation Function (PACF) values:")
for lag, pacf_val in enumerate(pacf_values):
    print(f"Lag {lag}: {pacf_val}")
```

## Action 09: **Calculating Partial Autocorrelation**

Partial Autocorrelation Function (PACF) values:

Lag 0: 1.0

Lag 1: -0.14660091544107987

Lag 2: -0.41670783556551094

Lag 3: -0.2542677161630039

Lag 4: -0.22850249203285483

Lag 5: -0.2243194641517822

Lag 6: -0.15154675491645925

Lag 7: -0.19308416231798248

Lag 8: 0.009427360424904765

Lag 9: -0.040035011639930534

Lag 10: -0.10937823669154328

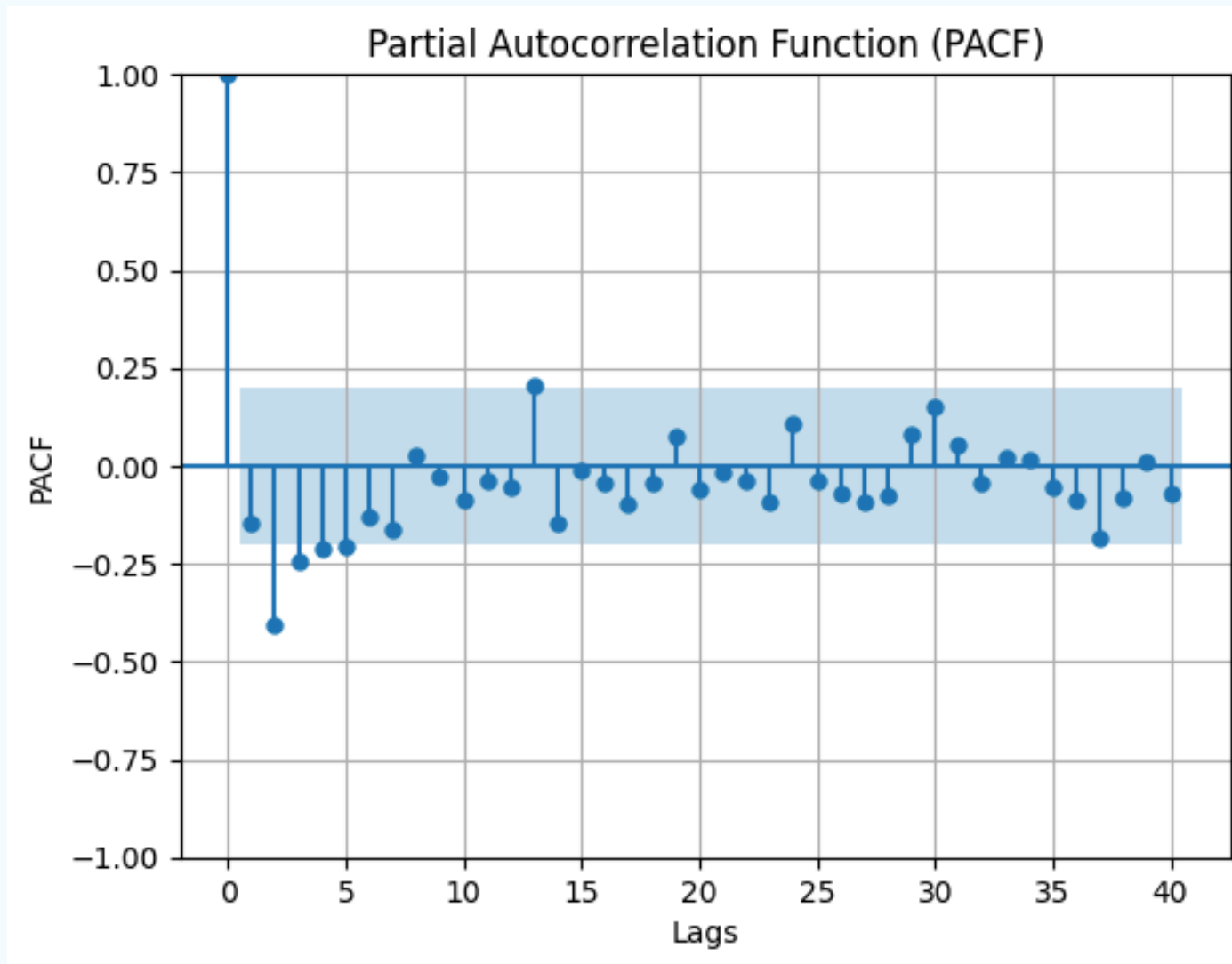
Lag 11: -0.0572427756557631

Lag 12: -0.07805508594341463

## Action 09: Calculating Partial Autocorrelation

```
# Plot PACF
plt.figure(figsize=(10, 5))
plot_pacf(CPI_rate, lags=40)
plt.title('Partial Autocorrelation
Function (PACF)')
plt.xlabel('Lags')
plt.ylabel('PACF')
plt.grid(True)
plt.show()
```

## Action 09: Calculating Partial Autocorrelation



## Action 10: **Calculate Moving Averages**

[Moving Averages](#), a statistical method in data analysis, smooths fluctuations in time-series data to reveal underlying trends. Calculating the average within a specified window and shifting it through the dataset, provides a clearer trend representation.

```
from google.colab import files
import pandas as pd
import io
import numpy as np
uploaded = files.upload()
```

## Action 10: Calculate Moving Averages

```
data = pd.read_csv('SolieuThuTSA001.csv')
arr = data['CPI_rate']
window_size = 3

# Convert array of integers to pandas series
numbers_series = pd.Series(arr)
# Get the window of series of observations of specified window size
windows = numbers_series.rolling(window_size)
# Create a series of moving averages of each window
moving_averages = windows.mean()

# Convert pandas series back to list
moving_averages_list = moving_averages.tolist()

# Remove null entries from the list
final_list = moving_averages_list[window_size - 1:]

print(final_list)
```



**THANK YOU**